

INTERNATIONAL CYBERSECURITY AND DIGITAL FORENSICS ACADEMY

Nasarawa State, Nigeria

LAB REPORT

Lab 02: PLC Programming Languages

Item	Details
Lab Title	Lab 02: PLC Programming Languages
Student Name	Sunusi Abdullahi
Student ID (Reg No)	C11/26/ACIS/17362
Course Name	Intro to Critical Infrastructure for IT Learners
Date of Submission	30/09/2026
Version	Version 1.0

1. Executive Summary

This laboratory introduced the different languages used to program a Programmable Logic Controller (PLC). A PLC is an industrial computer that controls machines such as motors, pumps and valves, and it can be programmed in more than one way. Working inside the isolated ICDFA OpenPLC virtual laboratory, I opened five example programs: Ladder Diagram, Function Block Diagram, Structured Text, Instruction List and Sequential Function Chart. All five describe the same simple task, which is to start a motor, keep it running, and stop it.

The main outcomes were these. In the Ladder Diagram I explained the rails, rungs, contacts and coils, and traced how the start, stop and latching logic works. I confirmed that the tags used are Boolean (true or false) values. I then inspected the other four languages and completed a comparison of their characteristics, uses and limits. It was shown that although the pictures and text look different, the control objective and the results are the same, so the meaning of each tag must never change between languages.

2. Lab Objectives

By the end of this lab, I should be able to:

- Identify the five PLC programming languages: Ladder Diagram, Function Block Diagram, Structured Text, Instruction List and Sequential Function Chart.
- Explain the purpose of Ladder Diagram rails, rungs, contacts and coils.
- Distinguish between normally open and normally closed contacts.
- Analyse a basic motor start/stop latching circuit.
- Examine the same motor-control requirement implemented in different PLC languages, and compare graphical and text-based approaches.
- Identify the PLC tags and Boolean data used in the motor-control program.
- Explain why different languages may suit different industrial automation tasks.

3. Tools and Resources Used

Tool / Resource	What it was used for
ICDFA PLC Virtual Machine (IP 192.168.1.10)	The training PLC environment supplied by ICDFA. All practical work was done inside it.
Virtualisation platform: (version)	Used to run the PLC VM from the clean snapshot on an isolated Host-Only network. Write the name and version you used.
Open PLC Editor (version)	Used to open the five example programs (LD, FBD, ST, IL and SFC) and the variable/tag list.
PLC Programming Language Examples folder	The folder holding the example projects for each language.
Screenshot tool	Used to save evidence for each language.

4. Methodology (Step-by-Step Procedure)

I followed the steps in the lab brief in order. One screenshot was captured at each important step, and every screenshot shows enough of the OpenPLC window (title bar and Project tree) to verify what was done. The screenshots are numbered Figures 1 to 8.

4.1 Evidence Map (Evidence 1 to 7)

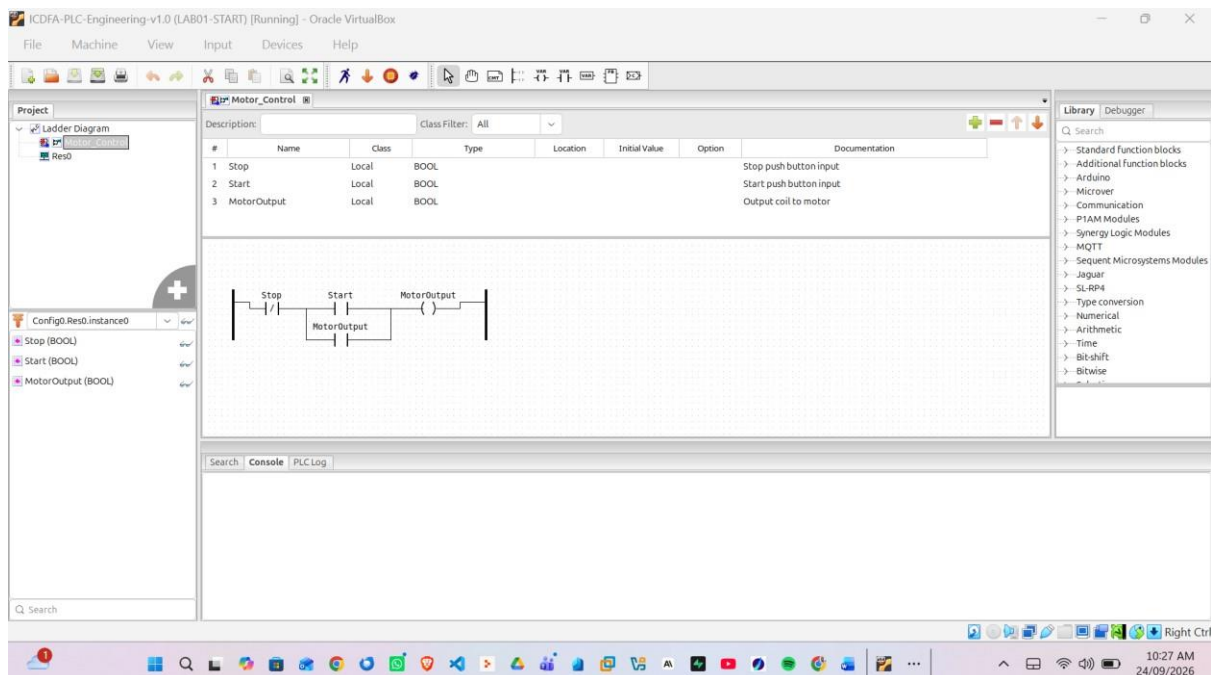
Every practical item in this report is labelled Evidence 1 to 7.

Evidence	What it covers	Figures
Evidence 1	The OpenPLC environment and the Ladder Diagram program	Figures 1, 2, 3
Evidence 2	Ladder Diagram parts, normally open / closed contacts, and the start/stop and latching logic	Figure 3
Evidence 3	PLC tags and Boolean data	Figure 4
Evidence 4	Function Block Diagram and Structured Text	Figures 5, 6
Evidence 5	Instruction List and Sequential Function Chart	Figures 7, 8
Evidence 6	The same requirement in five representations, with the same control objective	Figures 3 to 8
Evidence 7	Comparative analysis of the five languages	Section 5.7

4.3 The Steps

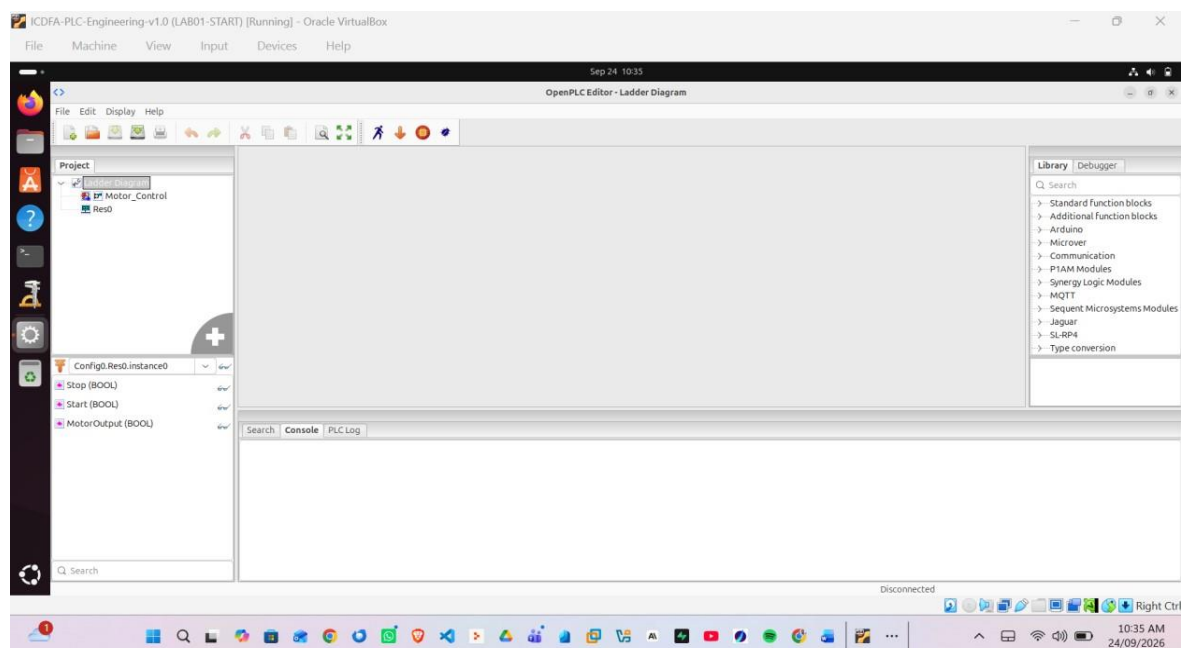
Step 1: Start the VM. I started from the clean ICDFFA PLC VM snapshot (LAB02-START, or the same clean snapshot used before), so the original stays safe. I signed in to the PLC training environment. I did not change any network setting and did not connect the VM to any production system.

Step 2: Launch OpenPLC Editor. I opened OpenPLC Editor. Then I clicked **File > Open** and browsed to the **PLC Programming Language Examples** folder, which holds the five language examples.



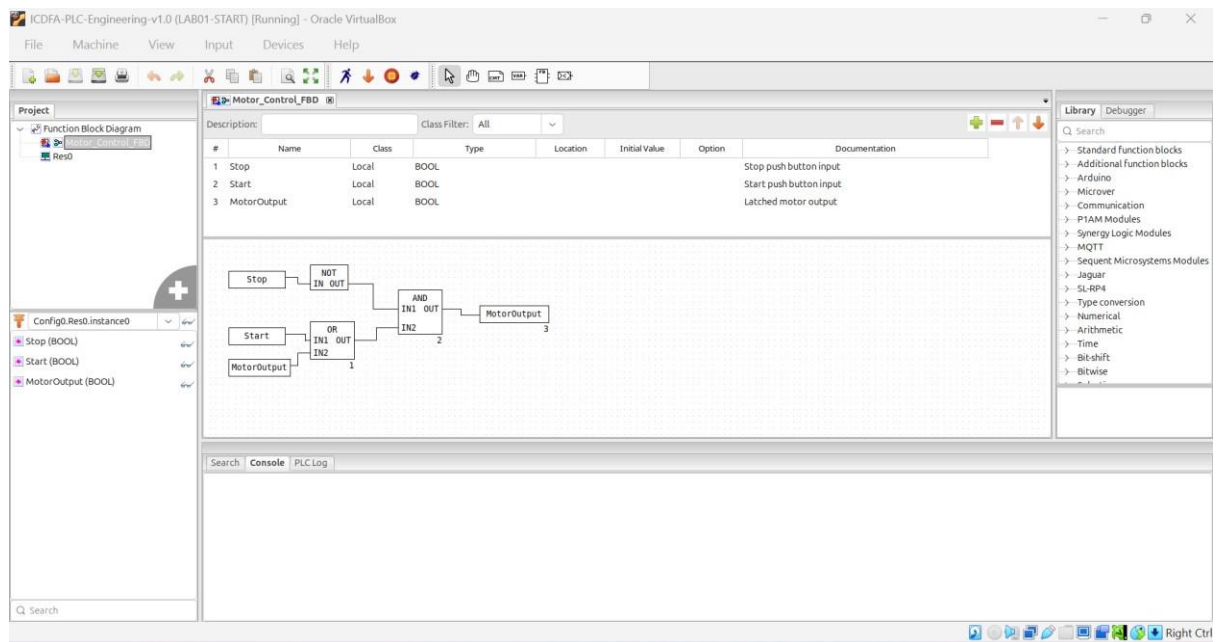
the **Ladder Diagram** example. In the Project tree, I opened **Res0** and then **instance0 (Motor_Control)**.

Step 4: Open the tag / variable list. I opened the variable list of the program. I checked that every motor-control tag has the type BOOL (Boolean). I wrote the purpose of each tag in my worksheet (see Evidence 3).

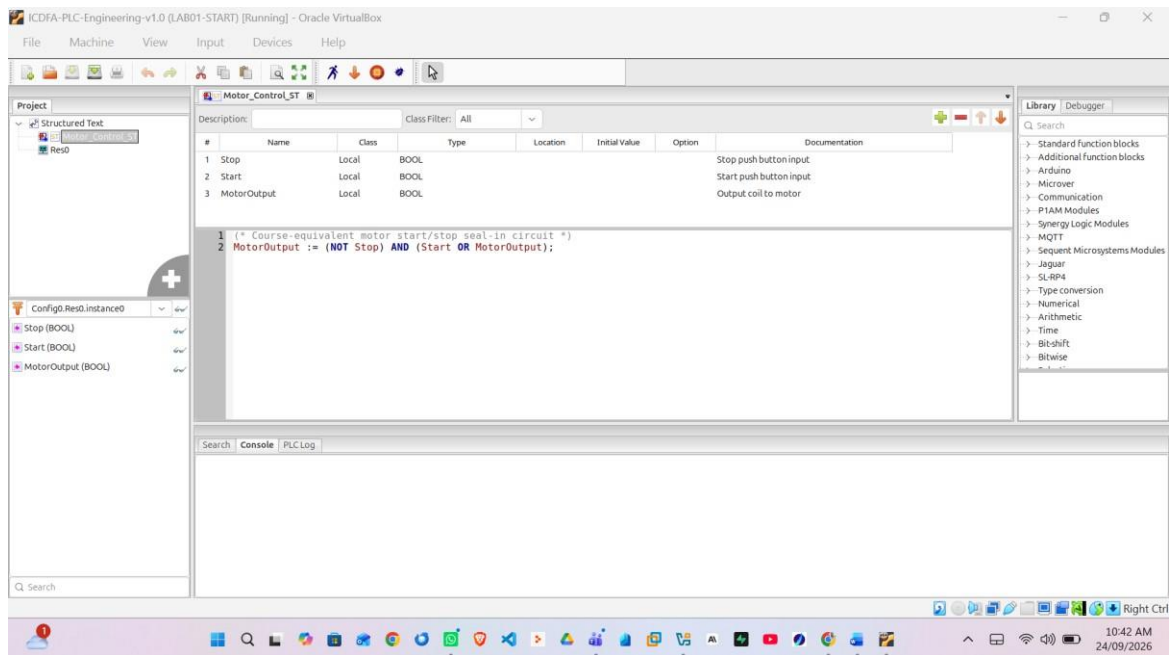


Step 5: Open the Function Block Diagram example. I clicked **File > Open** and loaded the Function Block Diagram example. I opened **Motor_Control_FBD**. I identified the block that keeps the motor output ON until a stop condition occurs.

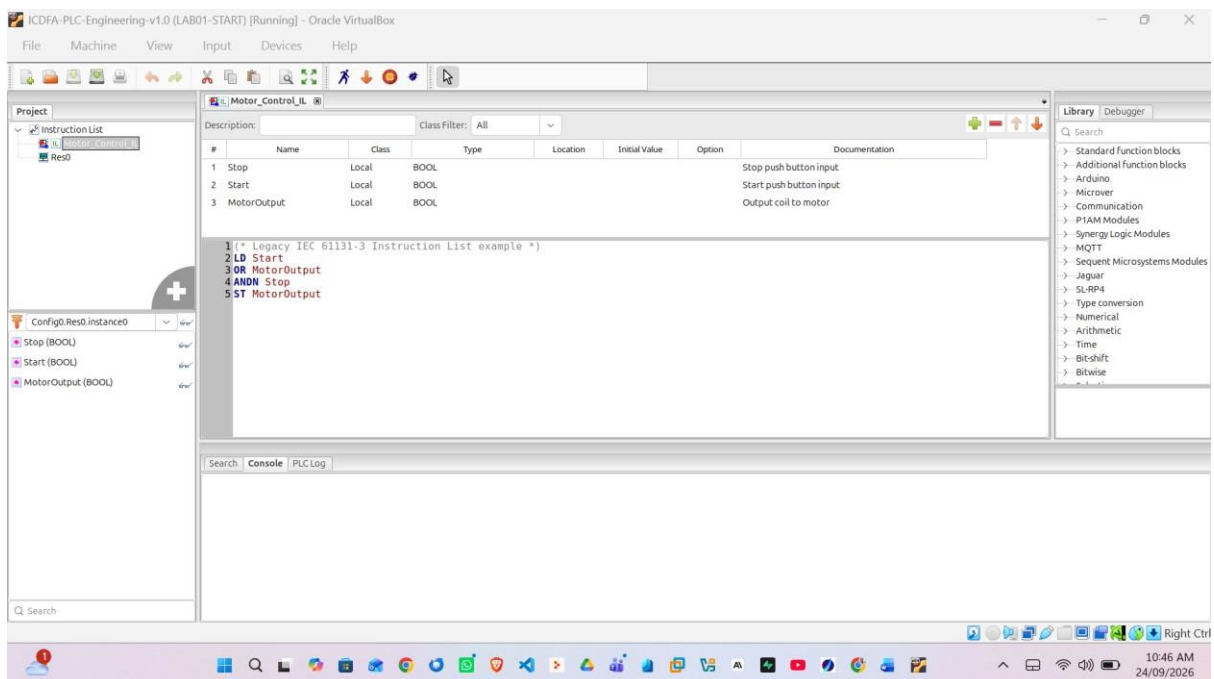
Step 6: Open the Structured Text example. I opened the Structured Text example and inspected **MotorControlST**. I noted how the same start/stop behaviour is written as text instead of contacts and coils.

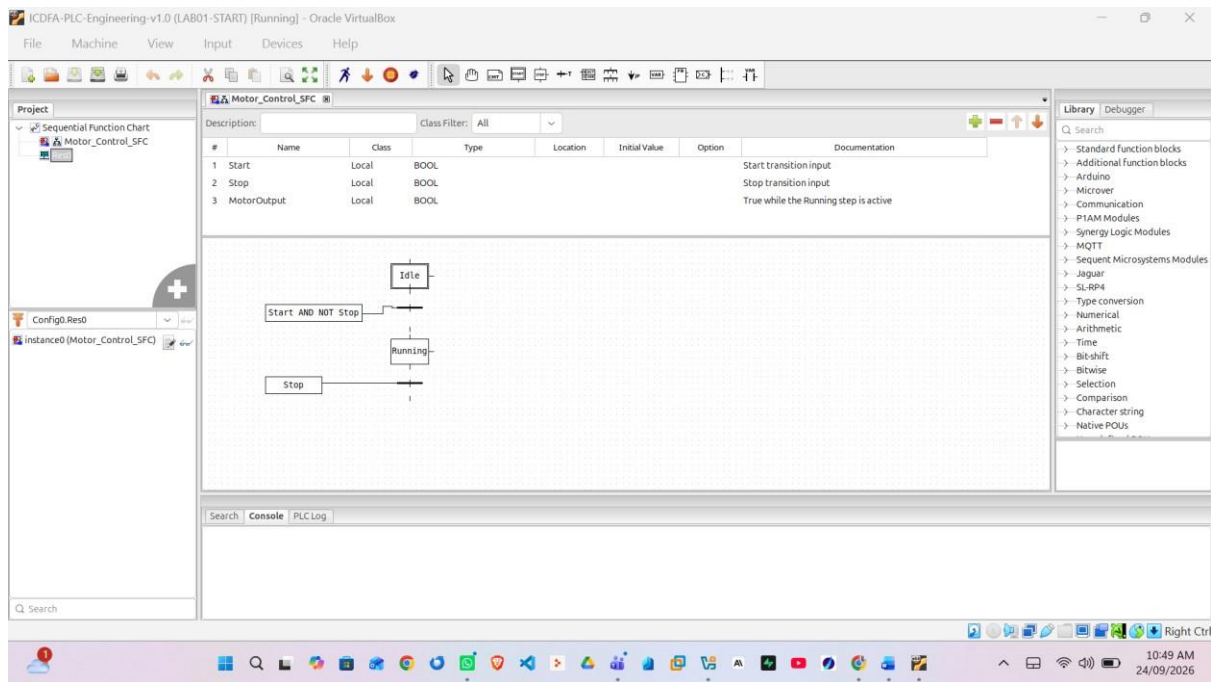


Step 7: Open the Instruction List example. I opened the Instruction List example and inspected **MotorControl**. I recorded the main visual difference between it and Structured Text.



Step 8: Open the Sequential Function Chart example. I opened the Sequential Function Chart example and inspected **MotorControl_SFC**. I identified the steps and the transitions that show the order of the process.





Step 9 and Step 10: Complete the comparison and finish. I completed the language comparison for all five languages (Evidence 7). I checked that each screenshot shows the program name. Then I saved my report and restored the VM to the clean snapshot / powered it off as directed by the instructor.

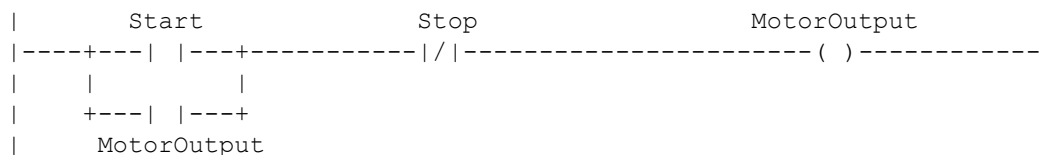
5. Evidence and Analysis

5.1 Evidence 1: The OpenPLC Environment and the Ladder Diagram Program

Question: Show the OpenPLC environment and the Ladder Diagram motor-control program (Figures 1, 2 and 3).

Figure	What it shows and what it means
Figure 1	The ICDFFA PLC VM was running from a clean snapshot in the authorised laboratory. This proves the work was done in the assigned environment.
Figure 2	OpenPLC Editor was opened and the PLC Programming Language Examples folder was found. This folder holds the same motor-control task written in the five languages.
Figure 3	The Ladder Diagram program Motor_Control (Res0 > instance0) was opened. It is the first of the five representations and the one used to explain the logic in Evidence 2.

Blackboard sketch of the Ladder Diagram (my simple drawing; the OpenPLC screen in Figure 3 is the real one):



Key: || = normally open (NO) contact, ||/ = normally closed (NC) contact, () = output coil.

Check on your screen: the names and positions may look slightly different in OpenPLC. Confirm that Stop is drawn with a slash (normally closed) and that the MotorOutput contact is drawn in parallel (under) with Start.

5.2 Evidence 2: Ladder Diagram Parts and the Start/Stop Latching Logic

Question: Explain rails, rungs, contacts and coils, tell normally open from normally closed contacts, and explain the start/stop and latching logic.

Part A: The parts of a Ladder Diagram

A Ladder Diagram looks like an electric circuit turned on its side. Power is imagined to flow from the left to the right.

Part	How it looks	Its purpose
Left power rail	Vertical line on the left	The "power in". Every rung starts from it.
Right power rail	Vertical line on the right	The "return". A rung is complete when power can reach it.
Rung	Horizontal line between the rails	One complete line of logic (one rule). It is solved from left to right.
Contact	or /	An input condition. It passes power or blocks it, depending on the value of its tag.
Coil	()	The output at the end of the rung. It switches ON when power reaches it.

Part B: Normally open and normally closed contacts

	Normally open (NO)	Normally closed (NC) /
Passes power when	its tag is TRUE (1).	its tag is FALSE (0).
Blocks power when	its tag is FALSE (0).	its tag is TRUE (1).
Simple picture	A gate that is shut until something opens it.	A gate that is open until something shuts it.
Used here for	Start button, and the MotorOutput latch.	Stop button.

Why is the Stop contact normally closed? Power passes through it all the time. Pressing Stop makes its tag TRUE, so the contact opens and cuts the path. It is also safer: if the Stop wire breaks, the circuit opens and the motor stops.

Part C: The elements of the motor circuit

Element	Symbol	Its job
Start contact	NO	Closes while the Start button is pressed. It gives the starting signal.
Stop contact	NC /	Conducts all the time. When the Stop button is pressed it opens and cuts the path.
MotorOutput contact (latch)	NO , parallel with Start	Copies the state of the coil. When the motor is ON it closes, and keeps the path complete after Start is released. This is called latching or seal-in.
MotorOutput coil	Coil ()	The output. When power reaches it, the coil is ON and the motor is ON.

Part D: Tracing the logic step by step

I trace the path from the left rail to the right rail for each situation. Here Stop = TRUE means the Stop button is pressed.

Case 1: Nothing is pressed. Start is open. The MotorOutput contact is open because the motor is OFF. There is no path to the coil. **Motor is OFF.**

Case 2: Press Start. Start closes. The Stop contact is normally closed, so it lets power pass. A complete path is made: left rail > Start > Stop > coil > right rail. **Motor is ON.** The MotorOutput contact closes too.

Case 3: Release Start. Start opens again. But the MotorOutput contact is now closed, so power still passes through the parallel branch: left rail > MotorOutput contact > Stop > coil > right rail. **Motor stays ON.** This is the latch.

Case 4: Press Stop. The normally closed Stop contact opens. The path is broken for both branches, so the coil loses power. **Motor is OFF.** The MotorOutput contact opens too.

Case 5: Release Stop. Stop closes again, but Start is open and the MotorOutput contact is open. There is still no path. **Motor stays OFF** until Start is pressed again.

Case	Start	Stop	MotorOutput contact (before)	Coil	Motor
1	Not pressed	Not pressed	Open	OFF	OFF
2	Pressed	Not pressed	Open	ON	ON
3	Released	Not pressed	Closed	ON	ON (latched)
4	Any	Pressed	Any	OFF	OFF
5	Released	Released	Open	OFF	OFF

Answer: The motor switches ON when Start is pressed (or the latch is already closed) AND Stop is not pressed. It switches OFF when Stop is pressed.

5.3 Evidence 3: PLC Tags and Boolean Data

Question: Identify the PLC tags and Boolean data used in the motor-control program and record the purpose of each tag (Figure 4).

Part A: What is a tag and what is Boolean?

Tag (variable): a name given to a memory location or an input/output of the PLC, for example Start. The program uses the name instead of a number address.

Boolean (BOOL): a data type that can only be TRUE (1) or FALSE (0). This matches ON/OFF devices such as buttons, lamps and motor contactors, so it is used for all discrete signals.

Part B: Tag worksheet

Write exactly what you see in the variable list of OpenPLC (Figure 4). If there are other tags, add more rows.

Tag name	Data type	Direction	Purpose
Start	BOOL	Input	Becomes TRUE when the Start push button is pressed. It gives the command to start the motor.
Stop	BOOL	Input	Becomes TRUE when the Stop push button is pressed. It gives the command to stop the motor.
MotorOutput	BOOL	Output	Controls the motor (TRUE = motor ON). It is also used as the latch contact so the program remembers that the motor is running.
_____	_____	_____	_____

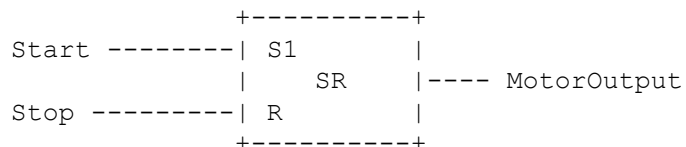
Conclusion: All the motor-control tags are BOOL because every signal in this program has only two states, ON or OFF.

5.4 Evidence 4: Function Block Diagram and Structured Text

Question: Show the Function Block Diagram and Structured Text implementations and explain how they do the same job (Figures 5 and 6).

Part A: Function Block Diagram (Motor_Control_FBD)

In a Function Block Diagram, ready-made **blocks** are joined by lines that show how signals flow. The block that keeps the motor ON until a stop condition occurs is a set-reset (SR) bistable: Start sets the output and Stop resets it. The block remembers its state, so it does the job of the latch.



Answer to Step 5: the block that maintains the motor output is the set-reset block. Write the exact block name you see in Figure 5: _____.

Check on your screen: write the exact name of the block you see (SR, RS or another) and how it is connected.

Part B: Structured Text (MotorControlST)

Structured Text is written like a short computer program. The same start/stop behaviour is written as one line:

```
MotorOutput := (Start OR MotorOutput) AND NOT Stop;
```

Word / symbol	Meaning in the motor circuit
:=	Assign. The result on the right is stored in MotorOutput. It does the job of the coil.
Start OR MotorOutput	The parallel branch: Start pressed OR the latch already closed.
AND	The two conditions are in series, so both must be true.
NOT Stop	The normally closed Stop contact: it passes only while Stop is not pressed.

Answer to Step 6: the same behaviour is written as text. The words OR and AND NOT take the place of the parallel contacts and the normally closed contact, and := takes the place of the coil.

5.5 Evidence 5: Instruction List and Sequential Function Chart

Question: Show the Instruction List and Sequential Function Chart versions and explain them (Figures 7 and 8).

Part A: Instruction List (MotorControl)

Instruction List is a low-level text language. Each line has one small instruction and one tag, like a very short order. The PLC keeps a "current result" and each instruction changes it.

```
LD      Start          (load Start)
OR      MotorOutput    (OR with the latch)
ANDN    Stop           (AND NOT Stop)
ST      MotorOutput    (store result in the motor output)
```

Working (Start = 1, Stop = 0, motor was OFF = 0):

Line 1: LD Start, so current result = 1.

Line 2: OR MotorOutput, so 1 OR 0 = 1.

Line 3: ANDN Stop, so 1 AND NOT 0 = 1 AND 1 = 1.

Line 4: ST MotorOutput, so the motor output = 1. **Motor ON.**

Answer to Step 7: IL is written as a list of short lines, one instruction and one tag per line (LD, OR, ANDN, ST), like assembly. Structured Text writes the same rule as one readable expression.

Check on your screen: the example in OpenPLC may be arranged slightly differently. Write what you see in Figure 7.

Part B: Sequential Function Chart (MotorControl_SFC)

A Sequential Function Chart shows the process as **steps** joined by **transitions**. A step is a state of the process, a transition is the condition to move to the next step, and an action is what happens in a step.

```
+-----+
| STOPPED | <- first step (motor OFF)
+-----+
      |
      +----- Transition: Start = TRUE
      |
+-----+
| RUNNING | --[ Action: MotorOutput ON ]
+-----+
      |
      +----- Transition: Stop = TRUE
      |
      go back to STOPPED
```

Answer to Step 8: the chart moves from STOPPED to RUNNING only when the Start transition is TRUE, and back to STOPPED only when the Stop transition is TRUE. So the steps and transitions show the order of the process.

Check on your screen: the step names and action in the OpenPLC example may be different. Write what you see in Figure 8.

5.6 Evidence 6: The Same Requirement in Five Representations

Question: Show how the same motor start/stop requirement is represented in different languages while the control objective stays the same.

Part A: The requirement

The motor must:

1. switch ON when the Start button is pressed;
2. stay ON after the Start button is released (latching);
3. switch OFF when the Stop button is pressed.

Simple Boolean rule:

$\text{MotorOutput} = (\text{Start OR MotorOutput}) \text{ AND NOT Stop}$

Part B: How each language shows the rule

Language	How the rule appears	Start / latch / Stop / output
Ladder Diagram (LD)	Contacts and a coil drawn between two rails.	Start NO contact in parallel with the MotorOutput NO contact, then a Stop NC contact, then the coil.
Function Block Diagram (FBD)	A set-reset block joined by lines.	Start to the set input, Stop to the reset input, the block output is MotorOutput.
Structured Text (ST)	One line of text.	$\text{MotorOutput} := (\text{Start OR MotorOutput}) \text{ AND NOT Stop};$
Instruction List (IL)	Four short instruction lines.	LD Start, OR MotorOutput, ANDN Stop, ST MotorOutput.
Sequential Function Chart (SFC)	Two steps and two transitions.	STOPPED to RUNNING on Start; RUNNING to STOPPED on Stop; action MotorOutput ON.

Part C: Checking that all five give the same result

I test the rule with values (1 = TRUE, 0 = FALSE). Rule: MotorOutput = (Start OR MotorOutput) AND NOT Stop.

Test 1: Start = 1, Stop = 0, motor was OFF (0). $(1 \text{ OR } 0) = 1$. $\text{NOT } 0 = 1$. $1 \text{ AND } 1 = 1$. **Motor ON.**

Test 2: Start released (0), Stop = 0, motor was ON (1). $(0 \text{ OR } 1) = 1$. $\text{NOT } 0 = 1$. $1 \text{ AND } 1 = 1$. **Motor stays ON (latched).**

Test 3: Stop pressed (1), motor was ON (1). $(0 \text{ OR } 1) = 1$. $\text{NOT } 1 = 0$. $1 \text{ AND } 0 = 0$. **Motor OFF.**

Test 4: Nothing pressed (0, 0), motor was OFF (0). $(0 \text{ OR } 0) = 0$. $\text{NOT } 0 = 1$. $0 \text{ AND } 1 = 0$. **Motor stays OFF.**

Start	Stop	MotorOutput (before)	MotorOutput (after)	Result
1	0	0	1	Motor starts
0	0	1	1	Motor stays ON
0	1	1	0	Motor stops
0	0	0	0	Motor stays OFF

Answer: LD, FBD, ST and IL all follow the same rule, and SFC gets the same result by moving between STOPPED and RUNNING. So the way it is written changes, but the control objective (start, hold, stop) stays the same. The tags Start, Stop and MotorOutput mean the same thing in every language.

5.7 Evidence 7: Comparative Analysis of the Five Programming Languages

Question: Explain the characteristics, applications and differences of the five PLC programming approaches.

Part A: Comparison matrix

Simple way to remember: LD looks like a wiring diagram. FBD looks like a flow of boxes. ST looks like a sentence. IL looks like a list of small orders. SFC looks like a road map of steps.

Language	Representation	Strengths	Limitations	Suitable industrial use case
Ladder Diagram (LD)	Graphical. Contacts and coils drawn between two power rails, like a relay wiring diagram.	Easy for electricians and technicians who know relay logic. Power flow is easy to see, so faults are easy to find.	Big or complex logic becomes long and crowded. Not good for heavy calculations.	Motor start/stop, interlocks, conveyor control.
Function Block Diagram (FBD)	Graphical. Ready-made blocks joined by lines that show how signals flow.	Shows data flow clearly. Reuses blocks such as timers, counters and PID.	Large diagrams get messy with many crossing lines. Less familiar to relay-logic technicians.	Process control, such as PID control of temperature, flow or level.
Structured Text (ST)	Textual. High-level code with :=, IF, AND, OR, NOT.	Short and compact. Best for maths, complex conditions and loops.	Needs programming skill. Harder to read at a glance for technicians. No visual power flow.	Calculations, data handling, recipes, complex algorithms.
Instruction List (IL)	Textual. Low-level, one instruction per line (LD, OR, ANDN, ST), like assembly.	Very compact and fast. Close to machine level. Good for small programs.	Hard to read and easy to make mistakes. Difficult to maintain. Now marked as deprecated in IEC 61131-3.	Small simple logic on small or older PLCs, and old (legacy) programs.
Sequential Function Chart (SFC)	Graphical. Steps joined by transitions to show the order of the process.	Shows the sequence clearly. Easy to see which step the machine is in.	Only shows the order. The actions and conditions still need LD, ST or another language. Too much for simple ON/OFF logic.	Batch processes, bottle filling lines, washing cycles, traffic light sequences.

Part B: Graphical versus text-based approaches

	Graphical (LD, FBD, SFC)	Text-based (ST, IL)
How it looks	Pictures: contacts, blocks, steps and lines.	Lines of text with words and symbols.
Strength	The structure and flow can be seen at once. Easy for technicians to read and to troubleshoot.	Compact. Handles maths, long conditions and loops in a few lines.
Weakness	Large programs take a lot of screen space and can look crowded.	Needs programming skill. The logic is not seen as a picture.
Best for	Machine logic, interlocks, process control, sequences.	Calculations, data handling and complex decisions.

Part C: Why different languages suit different tasks

Task	Best language	Reason
Simple interlock or motor start/stop	Ladder Diagram	Easy to see and to troubleshoot, like a relay circuit.
Continuous control such as PID for a tank level	Function Block Diagram	Ready-made blocks and clear signal flow.
Calculations, scaling, data processing	Structured Text	Compact and powerful for maths and logic decisions.
Very small, simple or legacy program	Instruction List	Compact and close to the machine (but now discouraged).
Batch or step-by-step process	Sequential Function Chart	Shows the order of stages and the current stage.

Conclusion of the comparison: No language is best for everything. Each one fits some tasks better. In real projects they are often mixed in one program. For example, an SFC can control the order of a batch while each step is written in LD or ST. What matters is that the control objective and the meaning of the tags stay the same.

5.8 Knowledge Check

1. Why is Ladder Diagram intuitive for technicians familiar with relay logic?

Ladder Diagram looks like the relay wiring diagrams that technicians already know. It uses contacts and coils between two power rails, and power flows from left to right like current in a circuit. So a technician can read it and find faults without learning a new way of thinking.

2. What is one advantage of Structured Text for complex calculations or conditions?

Structured Text is short and compact. Maths, long conditions, loops and IF/CASE decisions can be written in a few lines, while the same thing in ladder would need many rungs or blocks.

3. When might a Sequential Function Chart be more appropriate than a single ladder rung?

A Sequential Function Chart is better when the process has stages that must happen in order, for example filling, then mixing, then heating, then draining. A single ladder rung only shows the ON/OFF condition of one output. It does not show the order of the stages or which stage the machine is in.

4. Why must the meaning of tags remain consistent when the representation/language changes?

A tag is linked to a real device. If Start or Stop meant something different in another language, the same physical button would give a different command and the machine could behave wrongly or unsafely. Keeping the meaning the same also makes the programs easy to compare, translate and maintain.

7. Conclusion

This lab showed that the same PLC control task can be written in different programming languages. In the Ladder Diagram, rails, rungs, contacts and coils, with a normally open Start contact, a latching contact and a normally closed Stop contact, start, hold and stop the motor. Function Block Diagram shows signal flow, Structured Text is short and good for calculations, Instruction List is low-level and now old-fashioned, and Sequential Function Chart shows the order of a process. In all of them the motor starts on Start, stays on through the latch, and stops on Stop. The tags are Boolean and must keep the same meaning in every language.

8. Recommendations

- Choose the language that fits the task: LD for simple logic, ST for calculations, FBD for process control, and SFC for sequences.
- Keep tag names and meanings the same across all programs and write a short comment for each tag.
- Use a normally closed contact for Stop so that a broken wire stops the motor.

- Test every program only in an isolated training environment before using it on real equipment, and keep a clean snapshot of the VM.

9. References

International Cybersecurity and Digital Forensics Academy (ICDFA). (2026). Lab 02: PLC Programming Languages (Student Edition). Intro to Critical Infrastructure for IT Learners. <https://www.icdfa.edu.ng>

International Electrotechnical Commission (IEC). IEC 61131-3: Programmable controllers, Part 3: Programming languages.

OpenPLC Project. OpenPLC Editor documentation. <https://openplcproject.com>